Software Architecture and Design Specification

Project: Simplified GDPR Consent Manager
Version: 1.0
Authors: Team 6 - Vilas, Suman Diddimani, Pranaav Prabhakaran, Pradeep T
Date: 10-10-2026
Status: Final - submitted for review

Team Members

#	Name	SRN
1	VILAS	PES1UG24AM415
2	SUMAN DIDDIMANI	PES1UG24AM409
3	PRANAAV PRABHAKARAN	PES1UG24AM387
4	PRADEEP T	PES1UG24AM385

Revision History

Version	Date	Author	Change Summary	Approval
1.0	10-10-2026	Team 6	Architecture and design baseline derived from SRS v1.0	Submitted

Approvals

Role	Name	Signature / Date
Course Coordinator		

Table of Contents

1. Introduction
2. Document Overview
3. Architecture
4. Design
5. Appendices

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Simplified GDPR Consent Manager. It translates the requirements of SRS v1.0 (FR-1 to FR-20, SEC-1 to SEC-7, NFR-1 to NFR-14) into components, data structures, interfaces and design decisions that the development team will implement in Sprint 1 and Sprint 2.

1.2 Scope

The design covers the complete web application: the React single-page front end, the Node.js/Express REST API, the PostgreSQL data store, authentication and role-based access, consent recording, policy versioning, the tamper-evident audit trail, search and historical consent-state retrieval, deployment and CI. As stated in the SRS, the system is a simplified academic implementation and does not by itself guarantee legal GDPR compliance.

1.3 Audience

- Developers of Team 6 implementing the modules.
- QA/testers deriving test cases from the design.
- Project guide and course faculty evaluating the design.
- Reviewers assessing the security architecture.
- Future maintainers extending the system.

1.4 Definitions

Term	Meaning
ADR	Architecture Decision Record - a recorded design decision with its alternatives and rationale.
SPA	Single-Page Application (the React front end).
REST / JSON	HTTP API style used between front end and back end; payloads encoded as JSON.
JWT	JSON Web Token - signed token identifying the logged-in user and role.
RBAC	Role-Based Access Control (roles USER, ADMIN, DPO).
ORM	Object-Relational Mapper (Prisma) used to access PostgreSQL.
Hash chain	Each record stores the SHA-256 hash of the previous record, so any later modification is detectable.
STRIDE	Threat model: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
TLS	Transport Layer Security (HTTPS).
CI	Continuous Integration (GitHub Actions).

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals, stakeholders, the component and class diagrams, the chosen pattern with its Architecture Decision Records, the technology stack and database, risks, traceability to the SRS and the STRIDE security architecture. Section 4 describes the detailed design: sequence diagrams for the key flows, the REST API, error handling and logging, and the user-experience design. Developers should read Sections 3.3-3.6 and 4.2-4.4 before starting a backlog item; testers should use Section 3.8 together with the RTM in the SRS.

2.2 Related Documents

Document	Description	Location
SRS v1.0	Software Requirements Specification - requirements, use cases and RTM	docs/01-SRS/
SAD v1.0	This document	docs/02-SAD/
STP v1.0	Software Test Plan and test cases (TC-xxx IDs referenced here)	docs/03-Test-Plan/
Diagrams	Editable draw.io sources of all figures	docs/02-SAD/
Backlog	GitHub Project board with sprint items derived from the SRS	GitHub Projects

3. Architecture

3.1 Goals & Constraints

Architectural goals (with the SRS requirements that drive them):

Goal	How the architecture achieves it	Drivers
Integrity and traceability of consent	Append-only consent and audit tables, SHA-256 hash chain, every decision linked to a policy version.	FR-9, FR-11, FR-17, SEC-6
Security by default	Server-side RBAC on every endpoint, hashed passwords, httpOnly cookies, TLS, input validation.	SEC-1 to SEC-7
Responsiveness	Indexed queries, paginated search, single round-trip consent actions.	NFR-1 to NFR-3
Maintainability	Layered modules with one service per domain area; typed code; CI with tests on every PR.	NFR-12
Recoverability	Daily encrypted backups, tested restore, atomic transactions.	NFR-5, NFR-11
Simplicity for a 4-person team	Modular monolith, managed hosting, one database.	Course constraint

Constraints: two one-week sprints (19-23 Oct and 26-30 Oct 2026); open-source stack; free-tier hosting; minimal personal data (name, e-mail, password hash, role); all timestamps in UTC; no claim of legal GDPR certification.

3.2 Stakeholders & Concerns

Stakeholder	Main concerns	Addressed by
User (data subject)	Easy, honest opt-in/opt-out; seeing own status and history; privacy of own data	Consent Dashboard UX, ownership checks, FR-7/8/12
Administrator	Managing purposes, policies and users safely; reviewing admin activity	Admin Console, PolicyService, audit trail
DPO / Auditor	Trustworthy records, search, point-in-time state, tamper evidence	QueryService, hash-chain verification, read-only role
Developers (Team 6)	Clear module boundaries, quick setup, parallel work	Layered structure, Docker Compose, Prisma migrations
Course faculty / evaluators	Traceability from SRS to design to tests; working demo	Section 3.8, RTM, deployed demo

3.3 Component (UML) Diagram

Figure 1 is the UML component view of the layered (3-tier) architecture. The React web app uses the REST API provided by the Backend API subsystem, which contains one module per feature area of the SRS. Supporting components connect through provided (ball) and required (socket) interfaces; all data is stored in PostgreSQL.

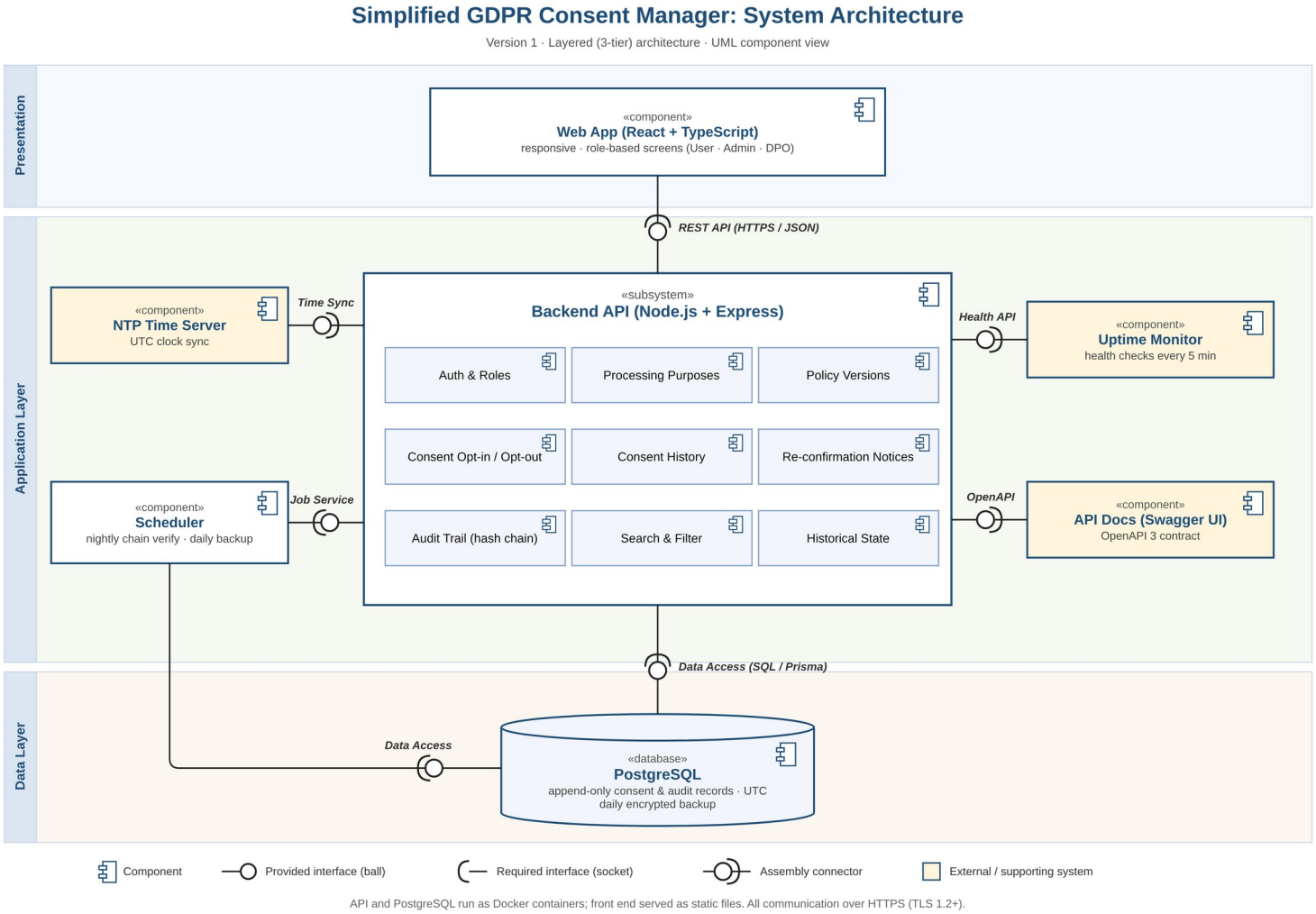

Figure 1. System Architecture - UML Component View.

3.3.1 Class (Domain Model) Diagram

Figure 2 shows the domain entities persisted in PostgreSQL and the main service classes. ConsentRecord and AuditEntry are immutable and hash-chained; the current and historical consent state is derived from ConsentRecord.

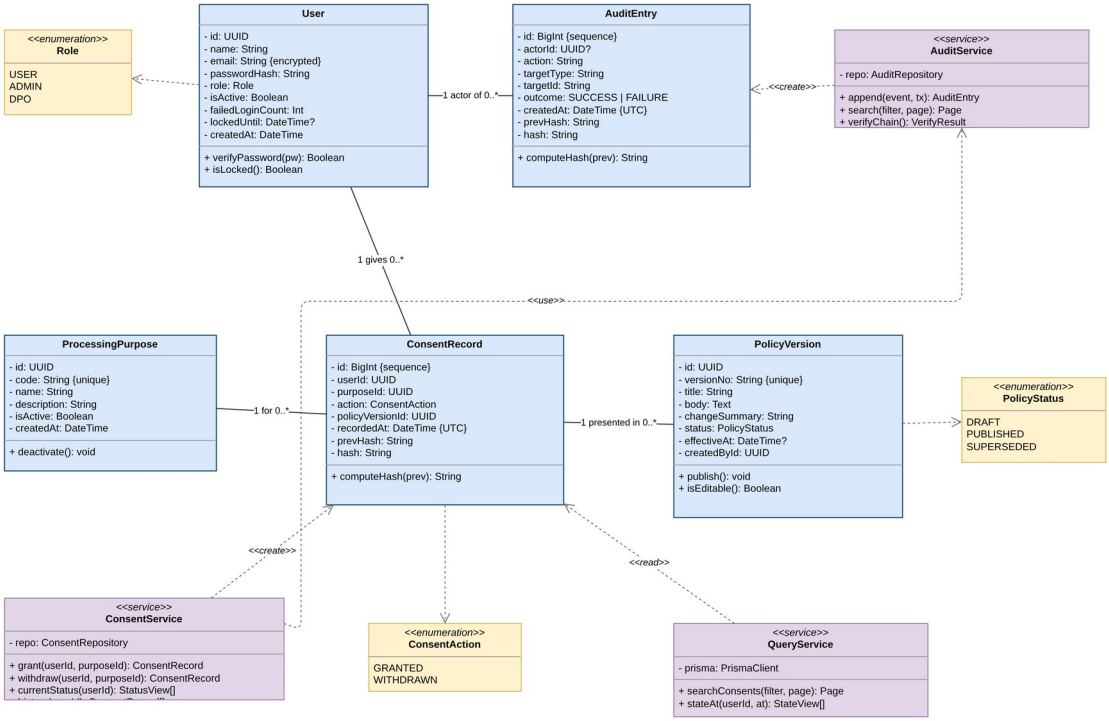

Figure 2. Class (Domain Model) Diagram.

3.4 Component Descriptions

Component	Responsibility	Key requirements
Web App (React + TypeScript)	Responsive SPA with role-based screens: Login/Register, Consent Dashboard, Policy Viewer, Consent History, Admin Console, DPO Console. Calls the REST API.	FR-1-5, FR-12, NFR-6, NFR-7
Backend API (Node.js + Express)	REST subsystem. Request pipeline: security headers and rate limiting, JWT authentication, RBAC guard, Zod validation, central error handler.	SEC-2, SEC-5, SEC-7
Auth & Roles	Register, login, logout, lockout after 5 failures, sessions, user and role administration.	FR-1-3, SEC-1, SEC-3, SEC-4
Processing Purposes	Create, edit, deactivate purposes; list active purposes.	FR-4, FR-6
Policy Versions	Draft, edit, publish and supersede policy versions; protect published versions.	FR-5, FR-13-15
Consent Opt-in / Opt-out	Grant and withdraw with validation; stores timestamp and policy version.	FR-7-10
Consent History	Append-only consent records; current status and own history.	FR-4, FR-11, FR-12
Re-confirmation Notices	Created when a new policy is published; shown to affected users at login.	FR-16
Audit Trail (hash chain)	Appends SHA-256 hash-chained audit entries in the same transaction; verifies the chain.	FR-17, FR-18, SEC-6
Search & Filter	DPO search of consent records and audit entries with filters and pagination.	FR-18, FR-19, NFR-2
Historical State	Consent state of a user at any past instant.	FR-20
NTP Time Server	Keeps the server clock accurate so all UTC timestamps are correct.	NFR-13
Scheduler	Nightly hash-chain verification and daily encrypted database backup.	SEC-6, NFR-11
Uptime Monitor	Calls GET /health every 5 minutes to measure availability.	NFR-4
API Docs (Swagger UI)	Publishes the OpenAPI 3 contract used by the front end and testers.	NFR-12
PostgreSQL	Single source of truth; append-only triggers; UTC timestamps; daily backups.	FR-11, NFR-8, NFR-11

3.5 Chosen Architecture Pattern and Rationale

Pattern: a layered, modular monolith - a React SPA (presentation) calling one Express REST API (application) backed by one PostgreSQL database (data). Inside the API, code is organised by domain module (auth, purposes, policies, consents, audit, query), each with its own routes, controller, service and tests. Consent is modelled in an event-sourced style: decisions are stored as immutable events and state is derived. The main decisions are recorded below.

ADR	Decision	Alternatives considered	Rationale
ADR-1	Layered modular monolith (SPA + REST API + one DB)	Microservices; serverless functions	Small team and two sprints; one deployable unit is simpler to build, test and demo. Modules keep boundaries clear.

ADR	Decision	Alternatives considered	Rationale
ADR-2	PostgreSQL relational database	MongoDB; SQLite	ACID transactions for consent + audit writes, foreign keys, triggers, and DISTINCT ON for point-in-time queries.
ADR-3	Append-only consent events; status derived from latest event	Mutable status column per user/purpose	Satisfies FR-11 (never overwrite) and makes FR-20 (historical state) a simple query.
ADR-4	SHA-256 hash chain on consent and audit records	Plain log table; external blockchain	Tamper evidence at negligible cost; verifiable with one endpoint; no extra infrastructure.
ADR-5	JWT in httpOnly, Secure, SameSite=Strict cookie + CSRF token	Token in localStorage; server sessions	Not readable by injected scripts; stateless API; sliding 15-minute expiry gives idle timeout (SEC-3).
ADR-6	Prisma ORM; raw SQL only for the point-in-time query	Hand-written SQL; TypeORM	Type-safe queries and migrations; parameterisation by default (SEC-7).

3.6 Technology Stack & Data Stores

Layer	Technology	Purpose
Front end	React 18, TypeScript 5, Vite, Tailwind CSS, React Router, Axios	Responsive SPA with role-based screens
Back end	Node.js 20 LTS, Express 4, TypeScript 5	REST API
Validation / security	Zod, bcrypt, jsonwebtoken, Helmet, express-rate-limit, csurf-style double-submit token	Input validation, hashing, tokens, headers
Data access	Prisma ORM 5	Schema, migrations, typed queries
Database	PostgreSQL 16	Persistent store, triggers, transactions
Logging	pino (JSON logs) with request IDs	Diagnostics without personal data
API documentation	OpenAPI 3 + Swagger UI at /api/docs	Shared contract for front end and testers
Testing	Jest + Supertest (API), Vitest + React Testing Library (UI), k6 (load)	Unit, integration, performance tests
DevOps	Docker Compose (local DB), GitHub Actions (lint, test, coverage on every PR)	Repeatable setup and CI
Hosting	Render (API), Vercel (front end), Neon (managed PostgreSQL, encrypted at rest)	Free-tier live demo over HTTPS

Database tables (all primary keys indexed; all timestamps timestamptz in UTC):

Table	Key columns	Notes
users	id, name, email_enc, email_hash (unique), password_hash, role, is_active, failed_login_count, locked_until, created_at	E-mail stored AES-256-GCM encrypted; email_hash (HMAC) used for login lookup (NFR-10).
processing_purposes	id, code (unique), name, description, is_active, created_at, updated_at	Deactivated, never deleted (FR-6).
policy_versions	id, version_no (unique), title, body, change_summary, status, effective_at, created_by	Partial unique index allows only one PUBLISHED row; trigger blocks edits unless status = DRAFT (FR-15).
consent_records	id (sequence), user_id, purpose_id, action, policy_version_id, recorded_at, prev_hash, hash	Append-only: trigger rejects UPDATE/DELETE. Index (user_id, purpose_id, recorded_at DESC) for status and FR-20.
audit_entries	id (sequence), actor_id, action, target_type, target_id, outcome, created_at, prev_hash, hash	Append-only and hash-chained; indexes on created_at, actor_id, action (FR-19).
policy_review_notices	id, user_id, policy_version_id, created_at, acknowledged_at	Created on publish for users with consents under older versions; drives the re-confirmation notice (FR-16).

Hash-chain rule: hash = SHA-256(prev_hash + canonical JSON of the record's fields); the first record uses 64 zeros as prev_hash. Appends take a PostgreSQL transaction-level advisory lock so

two concurrent writes cannot fork the chain. GET /api/v1/audit/verify recomputes both chains and reports the first broken link, if any.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Team new to parts of the stack; only two sprints	Features slip	Starter skeleton and CI set up in the dry-run week; vertical slices; core items in Sprint 1.
Concurrent writes break the hash chain	False tamper alarms	Advisory lock per chain inside the transaction; automated chain-verification test.
Clock skew produces wrong timestamps	Wrong historical state	Database now() in UTC is the only time source; host clock synced with NTP (NFR-13).
Free-tier hosting sleeps or is unavailable	Demo fails, NFR-4 missed	/health endpoint, uptime monitor, warm-up before demos, Docker Compose local fallback.
Database loss or corruption	Loss of consent evidence	Daily encrypted backup and a tested restore (NFR-11).
Secrets committed to the repository	Account compromise	.env files git-ignored; GitHub secret scanning and Actions secrets.
Scope creep	Unfinished core	SRS is the baseline; changes go through PRs and the revision history.
Over-claiming legal compliance	Misleading users	Disclaimer in the UI footer and documentation.

3.8 Traceability to Requirements

Requirement	Component(s)	Design element / API
FR-1	AuthService	POST /auth/register; role defaults to USER
FR-2	AuthService, SPA routing	POST /auth/login; role-based home screen
FR-3	AuthService	POST /auth/logout clears cookie and revokes token id
FR-4	PurposeService, ConsentService	GET /purposes, GET /me/consents
FR-5	PolicyService	GET /policies/current
FR-6	PurposeService	POST /purposes, PATCH /purposes/{id}, PATCH /purposes/{id}/deactivate
FR-7, FR-8	ConsentService	POST /me/consents {action: GRANT | WITHDRAW} - Figure 3
FR-9	ConsentService, ConsentRecord	user, purpose, action, recorded_at (UTC), policy_version_id stored
FR-10	ConsentService	409 NO_CHANGE, 422 PURPOSE_INACTIVE / NO_PUBLISHED_POLICY
FR-11	consent_records table	Append-only trigger; hash chain
FR-12	ConsentService	GET /me/consents/history
FR-13	PolicyService	POST /policies, PATCH /policies/{id} (DRAFT only)
FR-14	PolicyService	POST /policies/{id}/publish - Figure 4
FR-15	PolicyService, policy_versions trigger	GET /policies, GET /policies/{id}; edits refused unless DRAFT
FR-16	PolicyService, policy_review_notices	GET /me/notices, POST /me/notices/{id}/ack
FR-17	AuditService	append() called in every state-changing transaction
FR-18	AuditService, QueryService, RBAC	GET /consents, GET /audit (DPO); GET /audit?scope=admin (Admin)
FR-19	QueryService	Filters user, purpose, action, policyVersion, from, to; page, pageSize
FR-20	QueryService	GET /users/{id}/consent-state?at= - Figure 5
SEC-1	AuthService	bcrypt (cost 12); minimum 8-character password
SEC-2	RBAC Guard	Route role lists; ownership check on /me routes; 403 audited
SEC-3	AuthService, JWT middleware	Lock 15 min after 5 failures; 15-minute sliding token expiry
SEC-4	AuthService (user admin)	GET /users, PATCH /users/{id}/role, PATCH /users/{id}/deactivate
SEC-5	Hosting + Helmet	HTTPS only, HSTS, HTTP redirected
SEC-6	DB triggers, grants, AuditService	No update/delete paths; app DB role has INSERT/SELECT only; GET /audit/verify
SEC-7	Validation, Prisma, React, CSRF token	Zod schemas, parameterised SQL, output escaping, SameSite cookie + CSRF header
NFR-1 to NFR-3	QueryService, indexes	Composite indexes, pagination (max 100 rows), k6 load tests
NFR-4, NFR-11	Hosting, backups	/health, uptime monitor, daily backup, restore drill

Requirement	Component(s)	Design element / API
NFR-5	ConsentService, AuditService	Single DB transaction for record + audit entry
NFR-6, NFR-7	SPA	One-click grant/withdraw, WCAG 2.1 AA checks
NFR-8, NFR-9, NFR-10	Data tier, logger	Retention setting, minimal schema, redacted logs, e-mail encryption
NFR-12	Repository, CI	Layered modules; Jest coverage gate 70% in GitHub Actions
NFR-13	Data tier	timestamptz, DB now() in UTC, local-time display in SPA
NFR-14	RTM	Every requirement mapped to TC-xxx in the STP

3.9 Security Architecture

Threat modelling with STRIDE was applied to the data flow of Figure 1 (web app - backend API - database):

Threat	Example in this system	Mitigation	Req.
Spoofing	Stolen password or session token	bcrypt hashing, lockout, httpOnly Secure SameSite cookie, short sliding expiry, rate-limited login	SEC-1, SEC-3
Tampering	Editing an old consent or audit row in the database	Append-only triggers, INSERT/SELECT-only DB grants, SHA-256 hash chain with verification	SEC-6, FR-11
Repudiation	User denies having consented	Each decision stores UTC time and policy version and produces an audit entry in the same transaction	FR-9, FR-17
Information disclosure	User reads another user's data; data sniffed in transit; secrets in logs	Ownership checks, RBAC, TLS 1.2+, e-mail encryption, log redaction	SEC-2, SEC-5, NFR-9, NFR-10
Denial of service	Password brute force; very large search requests	Rate limiting, lockout, maximum page size, query timeouts	SEC-3, NFR-2
Elevation of privilege	USER calling an admin endpoint; forged role	Server-side RBAC on every route, role read from signed token and re-checked against DB for admin actions; only Admin changes roles (audited)	SEC-2, SEC-4

4. Design

4.1 Design Overview

The repository is a monorepo with two applications and shared documentation:

```
TEAM-6_Simplified-GDPR-Consent-Manager/
|-- client/                 React + Vite SPA
|   `-- src/ pages/ components/ api/ hooks/ routes/
|-- server/                 Express API
|   |-- prisma/             schema.prisma, migrations/, seed.ts
|   `-- src/
|       |-- middleware/     auth.ts, rbac.ts, validate.ts, errorHandler.ts
|       |-- modules/        auth/ purposes/ policies/ consents/ audit/ query/
|       |                   (each: routes.ts, controller.ts, service.ts, *.test.ts)
|       `-- lib/            hashChain.ts, crypto.ts, logger.ts
|-- docs/                   01-SRS/ 02-SAD/ 03-Test-Plan/
|-- docker-compose.yml      local PostgreSQL
`-- .github/workflows/ci.yml
```

Controllers are thin (parse request, call service, shape response). Services hold the business rules and open transactions. Each module is owned by one team member, which allows parallel work with one pull request per feature.

4.2 UML Sequence Diagrams

Figure 3 - Opt-in consent (FR-7, FR-9, FR-10, FR-17). The consent record and its audit entry are written in one transaction; an invalid action returns an error and writes nothing.

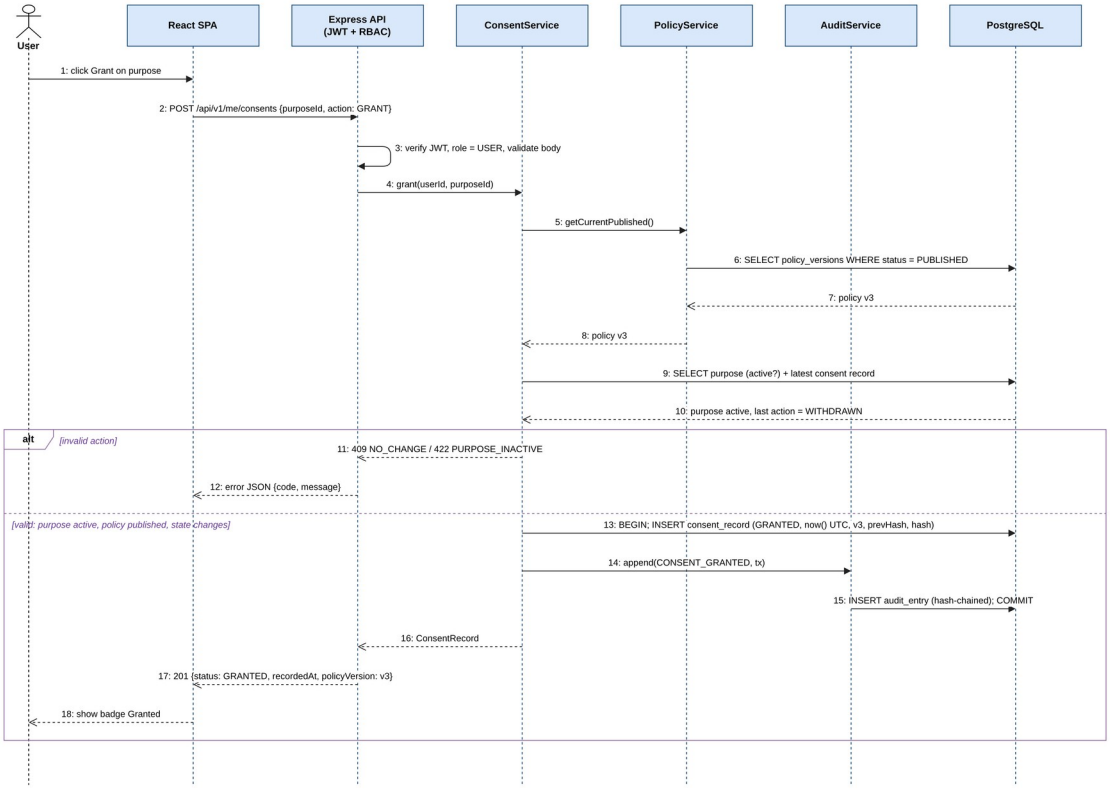

Figure 3. Sequence Diagram - Opt-in Consent.

Figure 4 - Publish policy version (FR-14, FR-15, FR-16). Publishing supersedes the current version, creates re-confirmation notices and is audited, all atomically.

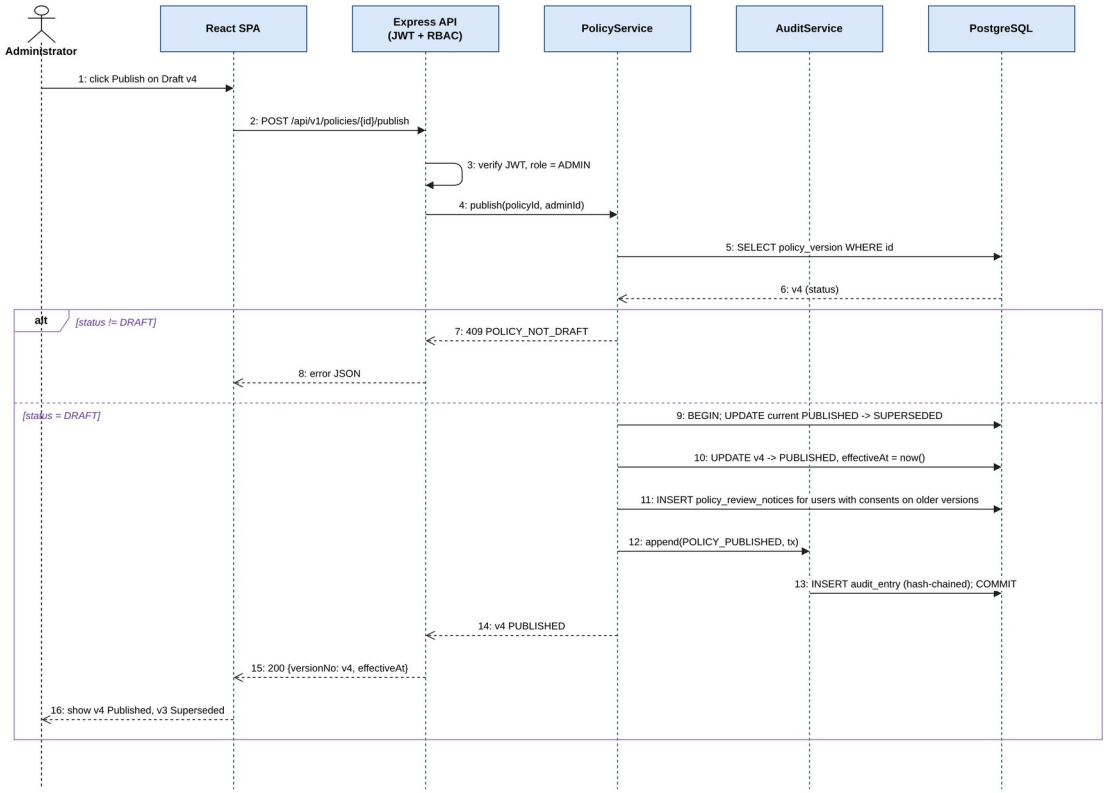

Figure 4. Sequence Diagram - Publish Policy Version.

Figure 5 - Historical consent state (FR-20). The state at instant t is the latest record per purpose with recorded_at <= t; purposes without a record are reported as Not given.

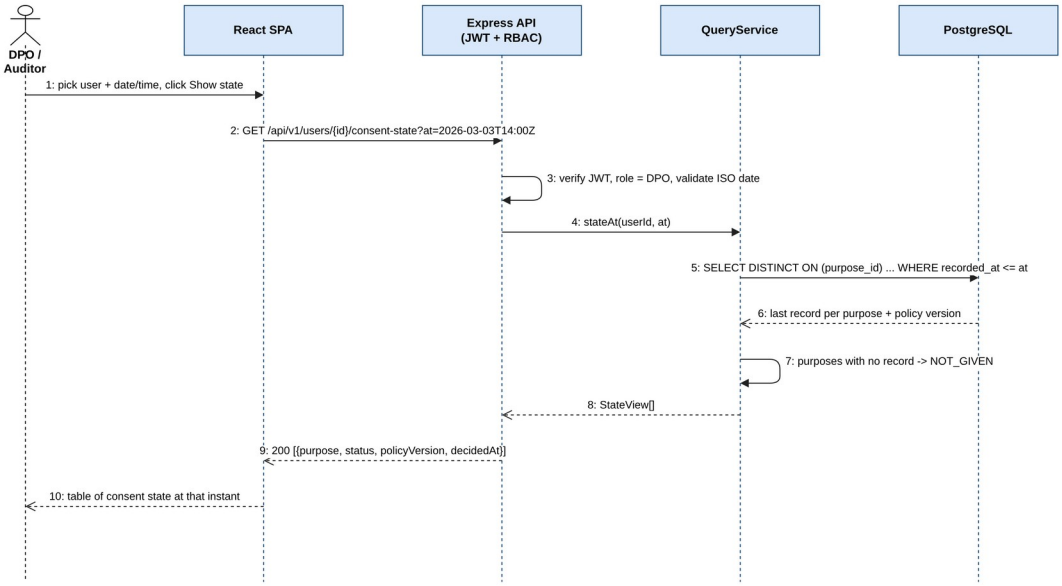

Figure 5. Sequence Diagram - Historical Consent State.

4.3 API Design

Base path /api/v1. All endpoints use JSON over HTTPS. Authentication uses the httpOnly cookie set at login; state-changing requests also send the X-CSRF-Token header. The full contract is published as OpenAPI 3 at /api/docs.

Method	Endpoint	Role	Purpose	Req.
POST	/auth/register	Public	Create account (role USER)	FR-1
POST	/auth/login	Public	Log in, set session cookie	FR-2, SEC-3
POST	/auth/logout	Any	End session	FR-3
GET	/auth/me	Any	Current user and role	FR-2
GET	/purposes	Any	Active purposes	FR-4
POST / PATCH	/purposes, /purposes/{id}, /purposes/{id}/deactivate	ADMIN	Manage purposes	FR-6
GET	/policies/current	Any	Published policy	FR-5
GET	/policies, /policies/{id}	ADMIN, DPO	List / view all versions	FR-15
POST / PATCH	/policies, /policies/{id}	ADMIN	Create / edit draft	FR-13
POST	/policies/{id}/publish	ADMIN	Publish draft	FR-14, FR-16
GET	/me/consents	USER	Current status per purpose	FR-4
POST	/me/consents	USER	Grant or withdraw	FR-7-10
GET	/me/consents/history	USER	Own history	FR-12
GET / POST	/me/notices, /me/notices/{id}/ack	USER	Policy re-confirmation notices	FR-16
GET	/consents	DPO	Search consent records	FR-18, FR-19
GET	/users/{id}/consent-state?at=	DPO	State at a point in time	FR-20
GET	/audit	DPO, ADMIN (scope=admin)	Search audit trail	FR-18, FR-19
GET	/audit/verify	DPO	Verify hash chains	SEC-6
GET / PATCH	/users, /users/{id}/role, /users/{id}/deactivate	ADMIN	User and role admin	SEC-4
GET	/health	Public	Liveness for monitoring	NFR-4

4.3.1 Consent component - POST /api/v1/me/consents

```
Request   {"purposeId": "7f3c...e1", "action": "GRANT"}    // or "WITHDRAW"
201       {"purposeId": "7f3c...e1", "status": "GRANTED",
           "recordedAt": "2026-10-20T09:15:02.481Z",
           "policyVersion": "v3", "recordId": 1042}
Errors    400 VALIDATION_ERROR      body invalid or unknown fields
          401 UNAUTHENTICATED       no/expired session
          403 FORBIDDEN             role is not USER
          409 NO_CHANGE             action equals current status
          422 PURPOSE_INACTIVE      purpose deactivated
          422 NO_PUBLISHED_POLICY   no policy version published yet
```

4.3.2 Policy component - POST /api/v1/policies/{id}/publish

```
Request   (no body)
200       {"id": "c21d...90", "versionNo": "v4", "status": "PUBLISHED",
           "effectiveAt": "2026-10-21T05:30:00.000Z",
           "superseded": "v3", "noticesCreated": 128}
Errors    401 UNAUTHENTICATED   403 FORBIDDEN (not ADMIN)
          404 NOT_FOUND         unknown policy id
          409 POLICY_NOT_DRAFT  only a DRAFT can be published
```

4.3.3 Query component - GET /api/v1/users/{id}/consent-state?at=2026-03-03T14:00:00Z

```
200     {"userId": "a91b...", "at": "2026-03-03T14:00:00Z", "state": [
          {"purpose": "MARKETING_EMAIL", "status": "GRANTED",
           "policyVersion": "v2", "decidedAt": "2026-02-11T08:02:10Z"},
          {"purpose": "ANALYTICS", "status": "NOT_GIVEN",
           "policyVersion": null, "decidedAt": null} ]}
Errors  400 VALIDATION_ERROR (bad ISO date)
        403 FORBIDDEN (not DPO)
        404 NOT_FOUND (unknown user)
```

4.4 Error Handling, Logging & Monitoring

Errors: a central Express error handler converts every failure into one JSON shape and never exposes stack traces in production:

```
{"error": {"code": "NO_CHANGE",
           "message": "Consent is already granted for this purpose.",
           "requestId": "b7e1..."}}
```

- Validation errors return 400 with field-level details; unexpected errors return 500 with a generic message and are logged.
- Logging: pino JSON logs with request ID, route, status and duration; passwords, tokens, cookies and e-mail addresses are redacted (NFR-9). Security events (failed logins, 403s, lockouts) are also written to the audit trail.
- Monitoring: GET /health (database ping) checked every 5 minutes by an uptime monitor (NFR-4); p95 latency and error rate computed from logs; a scheduled job runs the hash-chain verification nightly and logs any break.
- Transactions: consent and audit writes share one transaction, so a failure rolls back both (NFR-5).

4.5 UX Design

- Consent Dashboard: one card per active purpose with name, description, a status badge (Granted / Withdrawn / Not given) and a single Grant or Withdraw button of equal size and prominence - no pre-ticked boxes and no dark patterns (FR-7, FR-8).
- The current policy version and effective date are shown on the dashboard with a link to the full text (FR-5).
- After a policy update, a banner asks the user to review the new version and re-confirm or withdraw (FR-16).

- History: a reverse-chronological timeline with purpose, action, time (in local time zone) and policy version (FR-12).
- Admin Console: tabs for Purposes, Policy Versions (draft editor, publish button, version list), Users & Roles, Activity.
- DPO Console: filterable tables for consent records and the audit trail, a point-in-time state lookup and a 'Verify integrity' button showing the hash-chain result.
- Accessibility and layout: WCAG 2.1 AA contrast, keyboard navigation, labelled controls, responsive down to 360 px (NFR-7).
- A footer disclaimer states that this is a simplified academic system and not a legal compliance certification.

4.6 Open Issues & Next Steps

- E-mail notification when a new policy is published (currently shown on next login).
- Downloadable consent receipt (PDF) for users.
- Two-factor authentication for Admin and DPO accounts.
- Public consent-status API for other applications of the organisation (out of scope for v1).
- Internationalisation of the user interface.

5. Appendices

5.1 Glossary

ADR, API, CI, CSRF, DPO, GDPR, JWT, ORM, RBAC, REST, SPA, STRIDE, TLS, UTC - see Section 1.4 and SRS Section 1.4.

5.2 References

- Simplified GDPR Consent Manager - SRS v1.0, Team 6, 2026.
- ISO/IEC/IEEE 42010:2022 - Architecture description.
- OWASP Top 10 (2021) and OWASP Application Security Verification Standard 4.0.
- Regulation (EU) 2016/679 (GDPR), Article 7 - Conditions for consent.
- NIST SP 800-160 Vol. 1 - Systems Security Engineering.

5.3 Tools

- draw.io (diagrams.net) - all diagrams in this document; sources in docs/02-SAD/.
- Visual Studio Code, Git and GitHub (Projects, Pull Requests, Actions).
- Swagger UI / OpenAPI 3 for the API contract; Postman for manual API checks.
- Docker Desktop for the local database.